

ADC/DAC & Sensor Interfacing

Day 4

Day 4 — Embedded Systems 7-Day Study Series



7-Day Course Roadmap

Day	Topic
Day 1 ✓	Introduction — Architecture, GPIO, Memory, Protocols
Day 2 ✓	Interrupts, Timers & PWM
Day 3 ✓	UART, SPI, I2C — Deep Dive
Day 4 →	ADC/DAC & Sensor Interfacing
Day 5	RTOS Fundamentals (FreeRTOS)
Day 6	Power Management & Low-Power Design
Day 7	Bootloaders, OTA & System Design Project

1. ADC — Analog-to-Digital Converter

An **ADC** converts a continuous analog voltage into a discrete digital number. The two core specifications are **resolution** (number of bits) and **sampling rate** (samples per second). The **Nyquist theorem** states you must sample at least **2× the highest signal frequency** to reconstruct it.

Key ADC Parameters:

Parameter	Formula / Value	STM32 Typical	Impact
Resolution	2^N output codes	12-bit (0–4095)	Voltage precision
LSB Size	$V_{ref} / 2^N$	$3.3V / 4096 = 0.8mV$	Minimum measurable step
Sampling Rate	$1 / (T_{smp} + T_{conv})$	Up to 2.4 MSPS	Signal bandwidth
ENOB	Effective Number of Bits	~10.5 bits actual	Real-world accuracy
INL / DNL	Integral / Differential NL	±1–2 LSB	Linearity error
Reference Voltage	V_{ref} sets full-scale range	3.3 V or VDDA	Absolute accuracy

ADC Conversion Modes (STM32):

Mode	Trigger	Best For
Single conversion	Software/event	On-demand reads (button, command)
Continuous	Auto-repeat	Streaming data (audio, scope)
Scan mode	Sequences channels	Multi-sensor polling in one DMA burst
Injected mode	High-priority event	Emergency reads interrupting regular scan
Discontinuous	N channels per trig	Low-power periodic sampling

Oversampling for Higher Resolution:

```
// Hardware oversampling: 16x samples → gain 2 extra bits (14-bit from 12-bit ADC)
hadc1.Init.OversamplingMode = ENABLE;
hadc1.Init.Oversampling.Ratio = ADC_OVERSAMPLING_RATIO_16;
hadc1.Init.Oversampling.RightBitShift = ADC_RIGHTBITSHIFT_4; // divide by 16

// DMA continuous read into buffer
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buf, BUF_SIZE);
```

Anti-Aliasing & Input Conditioning:

- **Low-pass filter** before ADC input — RC filter with $f_c = F_{\text{sample}}/2$ prevents aliasing.
- **Input impedance** — source impedance must be low; use op-amp buffer if $> 1\text{ k}\Omega$.
- **Decoupling** — place $100\text{nF} + 10\mu\text{F}$ capacitors on VDDA pin close to MCU.
- **Guard ring** on PCB — surround analog traces with GND guard to reduce coupling noise.
- **Separate AGND / DGND** — join at single star point near MCU to avoid digital noise injection.

SAMPLING TIME

STM32 ADC sampling time must be long enough for the input capacitor to charge fully. For high source impedance, increase sampling cycles (e.g. 480 cycles on F4). Formula: $T_{\text{samp}} > (R_{\text{in}} + R_{\text{adc}}) \times C_{\text{adc}} \times \ln(2^{(N+1)})$

■ 2. DAC — Digital-to-Analog Converter

A **DAC** converts a digital value to an analog voltage. STM32 DACs are typically **12-bit** ($0-4095 \rightarrow 0\text{ V}$ to V_{ref}). They're used for audio output, waveform generation, and analog set-points. For MCUs without a hardware DAC, **PWM + RC filter** provides a low-cost alternative.

Feature	Hardware DAC	PWM + RC Filter (Pseudo-DAC)
Resolution	12-bit (4096 steps)	8–16 bit (timer-dependent)
Update rate	~1 MSPS	Limited by PWM frequency
Output noise	Low (filtered output)	Higher (ripple from PWM)
CPU load	Zero (DMA-driven)	Zero after timer setup
Cost/complexity	Built-in (if available)	Just a resistor + capacitor
Settling time	~1 μs	~5–10 $\times$ RC time constant

DMA Waveform Generation (Sine Wave on DAC1):

```
// Pre-computed 256-point sine table (12-bit, 0-4095)
uint16_t sine_lut[256]; // populate with sin() values

// Timer 6 triggers DAC at desired sample rate
HAL_TIM_Base_Start(&htim6);
HAL_DAC_Start_DMA(&hdac1, DAC_CHANNEL_1,
    (uint32_t*)sine_lut, 256, DAC_ALIGN_12B_R);

// Output frequency = TIM6_update_rate / 256 points
```

PWM-DAC FORMULA

PWM pseudo-DAC: $V_{\text{out}} = (\text{Duty}\% / 100) \times V_{\text{cc}}$. Use RC with $f_c \ll \text{FPWM}$. For 10 kHz PWM and $f_c = 100\text{ Hz}$: $R=15.9\text{k}\Omega$, $C=100\text{nF}$. Ripple $\approx V_{\text{cc}}/(\text{PWM}_{\text{res}} \times 2\pi \times f_c/\text{FPWM})$.

■ 3. Common Sensor Types & Interfacing

Sensors convert physical quantities into electrical signals — voltage, current, resistance, or frequency. Choosing the right interface (ADC, digital, pulse-count) and signal conditioning circuit is critical for accurate measurements.

Sensor	Output Type	Interface	Typical Range	Notes
NTC Thermistor	Resistance	ADC + Voltage div	−55 to +150 °C	Non-linear; use Steinhart-Hart
PT100 / PT1000	Resistance	ADC + Wheatstone	200 to +850 °C	Linear; use 4-wire for accuracy
DS18B20	1-Wire digital	GPIO (1-Wire)	−55 to +125 °C	12-bit; parasitic power possible
DHT11 / DHT22	Single-wire	GPIO (timed)	0–100% RH, −40–80 °C	Slow (1 Hz); use DHT22 for accuracy
BME280	I2C / SPI	I2C or SPI	Temp+Hum+Pressure	Highly accurate; built-in comp
MPU-6050	I2C digital	I2C	±16g, ±2000°/s	6-DOF IMU; DMP for fusion
HC-SR04	Pulse width	GPIO (timer IC)	2–400 cm	Measure echo pulse duration
LDR (CdS)	Resistance	ADC + Voltage div	1 kΩ–10 MΩ	Very non-linear; for presence only
Current (ACS712)	Analog voltage	ADC	±5/20/30 A	Hall-effect; 66–185 mV/A sensitivity
Strain Gauge	Resistance (ΔR)	ADC + Wheatstone	μΩ changes	Use instrumentation amp (INA128)

Voltage Divider for Resistive Sensors:

```
Vadc = Vcc × R_fixed / (R_sensor + R_fixed)
```

```
// NTC thermistor read and Steinhart-Hart conversion
uint32_t raw = HAL_ADC_GetValue(&hadc1);
float Vout = (raw / 4095.0f) * 3.3f;
float R_ntc = 10000.0f * Vout / (3.3f - Vout); // 10kΩ fixed
float T_inv = (1.0f/298.15f) + (1.0f/3950.0f)*logf(R_ntc/10000.0f);
float T_celsius = (1.0f / T_inv) - 273.15f;
```

VOLTAGE DIVIDER TIP

Choose $R_{\text{fixed}} = R_{\text{sensor}}$ at mid-range temperature for best linearity. For a 10kΩ NTC at 25°C, use $R_{\text{fixed}} = 10\text{k}\Omega$. This maximises dV/dT sensitivity at 25°C.

■ 4. Signal Conditioning & Amplification

Most real-world signals are small (mV range), noisy, or offset — they need **amplification**, **filtering**, and **level shifting** before reaching the ADC. Signal conditioning ensures the signal uses the full ADC input range for maximum resolution.

Circuit	Use Case	Key Component
Non-inverting amp	Amplify small sensor output	Op-amp (LM358, OPA2134)
Instrumentation amp	Differential/bridge sensors	INA128, AD8221
Low-pass RC filter	Remove HF noise before ADC	R + C ($f_c = 1/2\pi RC$)
Active low-pass filter	Higher-order, precise f_c	Op-amp Sallen-Key topology
Level shifter	5V sensor → 3.3V ADC	Resistor divider or TXB0104
Precision voltage ref	Stable Vref for ADC	LM4040, REF3025
Anti-aliasing filter	Prevent aliasing artefacts	LPF with $f_c = F_{\text{sample}}/2$

Non-Inverting Op-Amp Gain:

```
Gain (Av) = 1 + (Rf / Rg)

Example: Rf=9kΩ, Rg=1kΩ → Av = 10x
100mV sensor output → 1.0V ADC input (better use of 3.3V range)

// Map amplified ADC reading back to original value
float voltage_in = (adc_raw / 4095.0f * 3.3f) / 10.0f; // divide by gain
```

OP-AMP WARNING

Always check op-amp rail-to-rail capability. A standard LM358 on 3.3V can only output to ~1.5V max. Use rail-to-rail op-amps (MCP6001, OPA340) for single-supply 3.3V systems.

■ 5. Digital Filtering Techniques

Even with good hardware conditioning, ADC readings contain noise. Digital filters in firmware smooth readings, remove spikes, and extract meaningful signals without extra hardware cost.

Filter	Algorithm	CPU Cost	Latency	Best For
Moving Average	Sum last N samples / N	Low	N/2 samples	Slow sensors, noise
Exponential IIR	$y = \alpha \cdot x + (1-\alpha) \cdot y_{prev}$	Very Low	Variable	Streaming data, ADC
Median Filter	Sort N samples, take middle	Medium	N/2 samples	Spike/impulse removal
Kalman Filter	State estimation + covariance	High	~0	IMU, fused sensors
Butterworth IIR	Biquad sections (scipy design)	Medium	Fixed	Audio, precise fc
Debounce (digital)	Count stable readings	Minimal	Debounce ms	Buttons, encoders

EMA (Exponential Moving Average) — Most Common Choice:

```
// alpha = 0.1 → heavy smoothing; alpha = 0.9 → fast response
#define ALPHA 0.05f
float ema = 0;

void adc_update(float new_sample) {
    ema = ALPHA * new_sample + (1.0f - ALPHA) * ema;
}

// Cutoff frequency: fc = alpha * Fs / (2 * pi * (1-alpha))
```

Median Filter (Spike Removal):

```
uint16_t median_filter(uint16_t *buf, uint8_t n) {
    // Simple insertion sort then return middle
    uint16_t sorted[n];
    memcpy(sorted, buf, n*2);
    for(int i=1;i<n;i++) { /* insertion sort */ }
    return sorted[n/2];
}
```

SENSOR FUSION TIP

For IMU/AHRS applications, a complementary filter (gyro + accelerometer fusion) provides good attitude estimation with minimal CPU cost: $angle = 0.98 \cdot (angle + gyro \cdot dt) + 0.02 \cdot accel_angle$

6. Calibration Techniques

Raw ADC readings rarely equal the true physical quantity. **Calibration** maps raw counts to real-world units using known reference points, correcting for **offset**, **gain error**, and **non-linearity**.

Method	Points Needed	Corrects	Complexity
Offset correction	1 (zero ref)	Constant offset error	Trivial
Gain correction	1 (known input)	Scale/gain error	Trivial
2-point linear	2 references	Offset + gain	Easy
Multi-point linear	N points	Best-fit line (LSQ)	Moderate
Polynomial curve fit	N points	Non-linearity	Moderate
Lookup table (LUT)	Many points	Any shape	Memory cost

2-Point Linear Calibration (Most Common):

```
// Known: raw_low at physical_low, raw_high at physical_high
float gain = (phys_high - phys_low) / (float)(raw_high - raw_low);
float offset = phys_low - gain * raw_low;

// Apply to every reading:
float calibrated = gain * adc_raw + offset;

// Example: 0°C = ADC 410, 100°C = ADC 3600
// gain = 100/(3600-410) = 0.03135, offset = 0 - 0.03135*410 = -12.85
```

STORAGE TIP

Store calibration coefficients in flash (non-volatile) so they survive power cycles. On STM32, use the last flash page or dedicated EEPROM emulation via HAL_FLASH_Program.

7. Practice Exercises

#	Exercise	Diff	Time
1	Read NTC thermistor via ADC; convert to °C using Steinhart-Hart; UART print	■■	45 min
2	DMA continuous ADC on 3 channels (temp, light, pot); average + display	■■■	60 min
3	Generate sine wave on DAC using DMA + TIM6 trigger at 1 kHz output freq	■■■	60 min
4	Read MPU-6050 accel+gyro via I2C; apply complementary filter for angle	■■■■	90 min
5	2-point calibrate ADC with two known voltages; store coefficients in flash	■■■	60 min
6	HC-SR04 ultrasonic distance: timer input capture, median filter, UART	■■■	60 min

8. Quick-Reference Cheat Sheet

Concept / API	Key Point
ADC resolution	12-bit → 4096 steps; LSB = $V_{ref}/4096 \approx 0.8 \text{ mV}$ at 3.3 V
HAL_ADC_Start_DMA()	Triggers DMA fill; fires HAL_ADC_ConvCpltCallback on done
Oversampling (HW)	$4\times$ oversampling + 2-bit right-shift = 1 extra bit resolution
Nyquist theorem	$F_{sample} \geq 2 \times F_{max_signal}$ (e.g. 1 kHz signal → 2+ kHz ADC)

HAL_DAC_Start_DMA()	DMA-driven DAC; pair with TIM trigger for precise waveforms
EMA alpha selection	α close to 1 = fast (noisy); α close to 0 = slow (smooth)
Steinhart-Hart equation	$1/T = A + B \cdot \ln(R) + C \cdot (\ln(R))^3$ — for NTC linearisation
Instrumentation amp gain	$G = 1 + 50k\Omega/R_g$ — use INA128 for bridge/strain sensors
2-point calibration	gain = $\Delta\text{Physical}/\Delta\text{Raw}$; offset = phys_lo – gain × raw_lo
Flash storage (STM32)	HAL_FLASH_Unlock → HAL_FLASH_Program → HAL_FLASH_Lock

Day 4 Summary & Next Steps

■ Covered Today — Day 4	➡ Coming Up — Day 5
• ADC resolution, sampling, Nyquist theorem • Scan/continuous/DMA ADC modes • Oversampling for extended resolution • DAC waveform gen with DMA + timer • Sensor types: NTC, PT100, IMU, ultrasonic • Signal conditioning & op-amp circuits • Digital filters: EMA, median, Kalman intro • 2-point calibration & flash storage	• FreeRTOS tasks, scheduler, tick • Task states: running, ready, blocked, suspended • Queues for inter-task communication • Semaphores & mutexes (priority inversion) • Software timers & event groups • Stack sizing and stack overflow detection • Idle hook and tickless low-power mode • Porting FreeRTOS to STM32 with CubeMX

Study tip: Use STM32CubeIDE's live expression watch + breakpoint to inspect ADC raw values in real time. The STM32 ADC self-calibration (HAL_ADCEx_Calibration_Start) should be called once after power-up for best accuracy.